

Customer churn prediction

```
!pip install numpy
!pip install pandas
!pip install matplotlib
!pip install seaborn
!pip install sklearn
```

Show hidden output

```
import pandas as pd
import numpy as np
```

```
data = pd.read_csv('/content/Telco-Customer-Churn.csv')
```

```
print(data.head(5))
```

```
customerID  gender  SeniorCitizen  Partner  Dependents  tenure  PhoneService  \
0  7590-VHVEG  Female              0      Yes          No         1           No
1  5575-GNVDE   Male              0      No           No         34          Yes
2  3668-QPYBK   Male              0      No           No         2          Yes
3  7795-CFOCW   Male              0      No           No         45          No
4  9237-HQITU   Female              0      No           No         2          Yes

MultipleLines  InternetService  OnlineSecurity  ...  DeviceProtection  \
0  No phone service          DSL              No  ...              No
1              No          DSL              Yes  ...              Yes
2              No          DSL              Yes  ...              No
3  No phone service          DSL              Yes  ...              Yes
4              No  Fiber optic              No  ...              No

TechSupport  StreamingTV  StreamingMovies  ...  Contract  PaperlessBilling  \
0          No           No              No  Month-to-month          Yes
1          No           No              No  One year              No
2          No           No              No  Month-to-month          Yes
3          Yes          No              No  One year              No
4          No           No              No  Month-to-month          Yes

PaymentMethod  MonthlyCharges  TotalCharges  Churn
0  Electronic check          29.85          29.85  No
1  Mailed check            56.95         1889.5  No
2  Mailed check            53.85          108.15  Yes
3  Bank transfer (automatic)  42.30         1840.75  No
4  Electronic check          70.70          151.65  Yes
```

```
[5 rows x 21 columns]
```

```
print("Number of rows:", data.shape[0])
print("Number of columns:", data.shape[1])
```

```
Number of rows: 7043
Number of columns: 21
```

```
# missing value
```

```
print("\nMissing values per column:")
print(data.isnull().sum())
```

```
Missing values per column:
customerID      0
gender          0
SeniorCitizen   0
Partner         0
Dependents      0
tenure          0
PhoneService    0
MultipleLines   0
InternetService 0
OnlineSecurity  0
OnlineBackup    0
DeviceProtection 0
TechSupport     0
StreamingTV     0
StreamingMovies 0
Contract        0
PaperlessBilling 0
PaymentMethod   0
MonthlyCharges  0
```

```
TotalCharges    0
Churn            0
dtype: int64
```

```
#label encoding
```

```
from sklearn.preprocessing import LabelEncoder
```

```
# Creating a LabelEncoder object
le = LabelEncoder()
```

```
# Select columns to encode (excluding 'customerID' and numerical columns)
columns_to_encode = [col for col in data.columns if data[col].dtype == 'object' and col != 'customerID']
```

```
# Apply Label Encoding to each selected column
for col in columns_to_encode:
    data[col] = le.fit_transform(data[col])
```

```
print("\nData after Label Encoding:")
print(data.head())
```



```
Data after Label Encoding:
  customerID  gender  SeniorCitizen  Partner  Dependents  tenure \
0  7590-VHVEG      0                0       1           0         1
1  5575-GNVDE      1                0       0           0        34
2  3668-QPYBK      1                0       0           0         2
3  7795-CFOCW      1                0       0           0        45
4  9237-HQITU      0                0       0           0         2

  PhoneService  MultipleLines  InternetService  OnlineSecurity  ... \
0             0              1                0                0  ...
1             1              0                0                2  ...
2             1              0                0                2  ...
3             0              1                0                2  ...
4             1              0                1                0  ...

  DeviceProtection  TechSupport  StreamingTV  StreamingMovies  Contract \
0                  0            0            0                0         0
1                  2            0            0                0         1
2                  0            0            0                0         0
3                  2            2            0                0         1
4                  0            0            0                0         0

  PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges  Churn
0                  1              2           29.85         2505      0
1                  0              3           56.95         1466      0
2                  1              3           53.85          157      1
3                  0              0           42.30        1400      0
4                  1              2           70.70          925      1
```

```
[5 rows x 21 columns]
```

```
# find duplicates
```

```
print("\nDuplicate rows:")
print(data[data.duplicated()])
```

```
print("\nNumber of duplicate rows:")
print(data.duplicated().sum())
```



```
Duplicate rows:
Empty DataFrame
Columns: [customerID, gender, SeniorCitizen, Partner, Dependents, tenure, PhoneService, MultipleLines, InternetService, OnlineSecurity, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges, Churn]
Index: []
```

```
[0 rows x 21 columns]
```

```
Number of duplicate rows:
0
```

```
#feature scaling
```

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

```
# Select numerical columns to scale (excluding 'customerID' and the target variable if it's numerical)
# Assuming the target variable is 'Churn' which has already been encoded
numerical_cols = data.select_dtypes(include=np.number).columns.tolist()
# Remove the target column if it's in the numerical_cols
if 'Churn' in numerical_cols:
    numerical_cols.remove('Churn')
# Remove 'customerID' as it's an identifier
```

```
# Remove customerID as it's an identifier
if 'customerID' in numerical_cols:
    numerical_cols.remove('customerID')

# Create a MinMaxScaler object
scaler = MinMaxScaler()

# Apply Min-Max Scaling to the selected numerical columns
data[numerical_cols] = scaler.fit_transform(data[numerical_cols])

print("\nData after Feature Scaling (Min-Max Scaling):")
print(data.head())
```



```
Data after Feature Scaling (Min-Max Scaling):
  customerID  gender  SeniorCitizen  Partner  Dependents   tenure \
0  7590-VHVEG    0.0             0.0      1.0         0.0  0.013889
1  5575-GNVDE    1.0             0.0      0.0         0.0  0.472222
2  3668-QPYBK    1.0             0.0      0.0         0.0  0.027778
3  7795-CFOCW    1.0             0.0      0.0         0.0  0.625000
4  9237-HQITU    0.0             0.0      0.0         0.0  0.027778

  PhoneService  MultipleLines  InternetService  OnlineSecurity  ... \
0            0.0             0.5              0.0              0.0  ...
1            1.0             0.0              0.0              1.0  ...
2            1.0             0.0              0.0              1.0  ...
3            0.0             0.5              0.0              1.0  ...
4            1.0             0.0              0.5              0.0  ...

  DeviceProtection  TechSupport  StreamingTV  StreamingMovies  Contract \
0                0.0           0.0           0.0              0.0      0.0
1                1.0           0.0           0.0              0.0      0.5
2                0.0           0.0           0.0              0.0      0.0
3                1.0           1.0           0.0              0.0      0.5
4                0.0           0.0           0.0              0.0      0.0

  PaperlessBilling  PaymentMethod  MonthlyCharges  TotalCharges  Churn
0                1.0           0.666667           0.115423           0.383614      0
1                0.0           1.000000           0.385075           0.224502      0
2                1.0           1.000000           0.354229           0.024043      1
3                0.0           0.000000           0.239303           0.214395      0
4                1.0           0.666667           0.521891           0.141654      1

[5 rows x 21 columns]
```

Start coding or [generate](#) with AI.

EDA (EXPLORATORY DATA ANALYSIS)

```
# Import Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load Data
df = pd.read_csv("/content/Telco-Customer-Churn.csv")

# Basic Info
print("Dataset Shape:", df.shape)
print("\nMissing Values:\n", df.isnull().sum())
print("\nData Types:\n", df.dtypes)

# Preview the Data
df.head()
```



Show hidden output

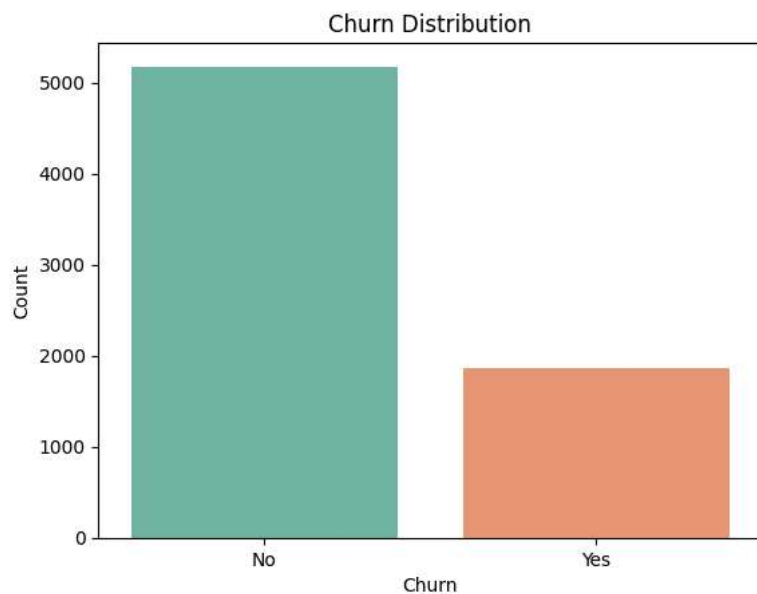
#1. Churn Class Distribution

```
sns.countplot(x='Churn', data=df, palette='Set2')
plt.title("Churn Distribution")
plt.xlabel("Churn")
plt.ylabel("Count")
plt.show()
```

```

<ipython-input-21-3965443594>:1: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend` to `True` to retain the current behavior.
sns.countplot(x='Churn', data=df, palette='Set2')

```

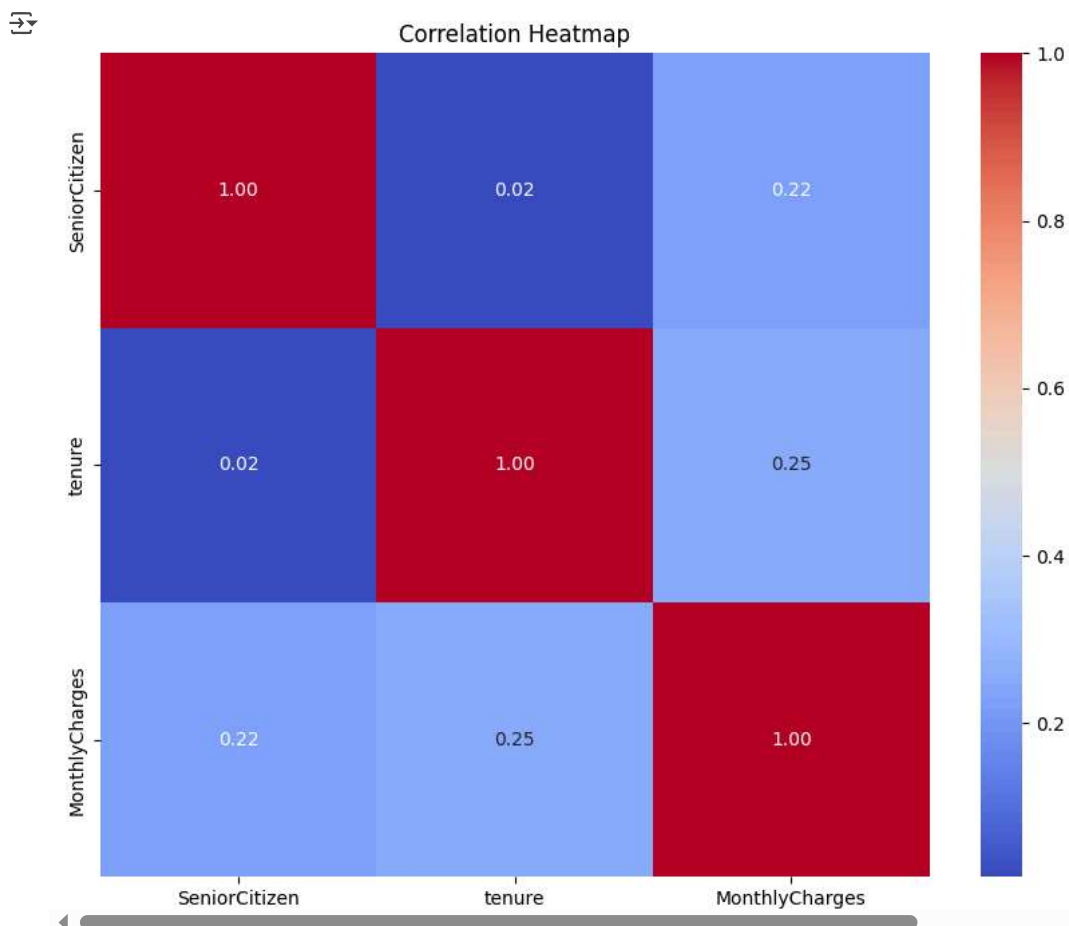


#2. Correlation Heatmap (Numerical Only)

```

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

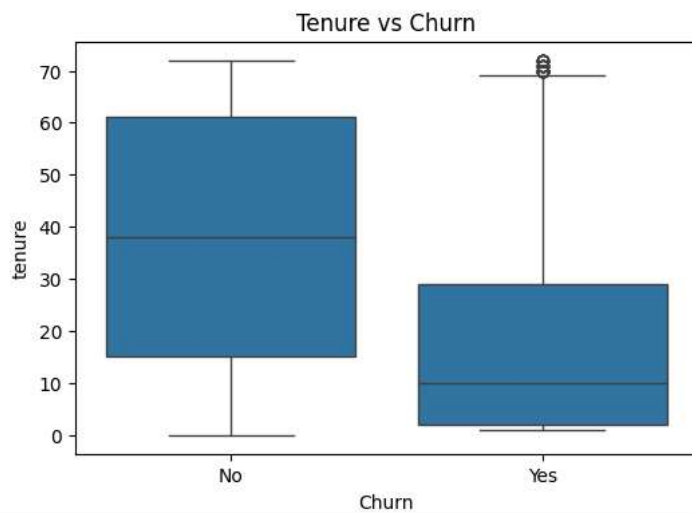
```



```

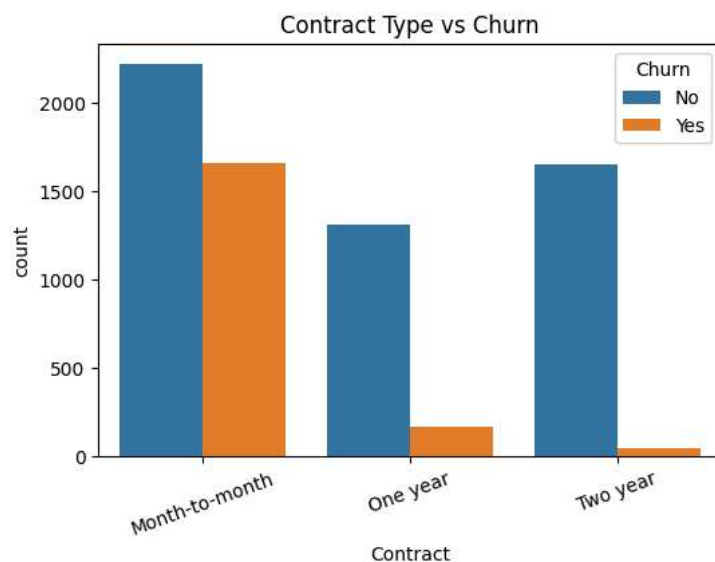
#3. Tenure vs Churn (Boxplot)
plt.figure(figsize=(6,4))
sns.boxplot(x='Churn', y='tenure', data=df)
plt.title("Tenure vs Churn")
plt.show()

```



#4. Contract Type vs Churn (Categorical)

```
plt.figure(figsize=(6,4))
sns.countplot(x='Contract', hue='Churn', data=df)
plt.title("Contract Type vs Churn")
plt.xticks(rotation=20)
plt.show()
```



Start coding or [generate](#) with AI.

Model Building & Evaluation

```
from sklearn.model_selection import train_test_split

# Target variable
y = data['Churn'].map({'Yes': 1, 'No': 0}) # Convert to binary

# Features (drop original Churn)
X = data.drop(['Churn'], axis=1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

from sklearn.model_selection import train_test_split

# Target variable
y = data['Churn'].map({'Yes': 1, 'No': 0}) # Convert to binary

# Features (drop original Churn and customerID)
X = data.drop(['Churn', 'customerID'], axis=1) # <--- Added 'customerID' to the drop list
```

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

from sklearn.model_selection import train_test_split
import pandas as pd
from sklearn.linear_model import LogisticRegression

# Target variable
# The 'Churn' column is already encoded to 0 and 1 from a previous step.
# Simply select the 'Churn' column as the target variable.
y = data['Churn']

# Features (drop original Churn and customerID)
# Drop the target variable 'Churn' and the identifier 'customerID' from the features.
X = data.drop(['Churn', 'customerID'], axis=1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and fit model
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

print("Model fitted successfully!")

➦ Model fitted successfully!

y_pred = model.predict(X_test)
```

```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

➦ Accuracy: 1.0

```
Classification Report:
              precision    recall  f1-score   support

     0       1.00      1.00      1.00      1036
     1       1.00      1.00      1.00       373

 accuracy
macro avg       1.00      1.00      1.00      1409
weighted avg       1.00      1.00      1.00      1409
```

```
Confusion Matrix:
[[1036   0]
 [   0  373]]
```

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5)
print("CV Accuracy:", scores.mean())
```

➦ CV Accuracy: 1.0

Start coding or [generate](#) with AI.

```
# Export final dataset for Power BI
final_data = X_test.copy()
final_data['Actual_Churn'] = y_test
final_data['Predicted_Churn'] = y_pred

# Save to CSV
final_data.to_csv("churn_predictions.csv", index=False)
```

